

An End-to-End Explainable Machine Learning Framework for Credit Card Fraud Detection with Deployment Performance Evaluation

Abhishek Kumar Purbey

Independent Researcher
abhiishek57@gmail.com

Abstract. Credit card fraud detection is a challenging machine learning problem due to severe class imbalance and the need to identify fraudulent transactions while limiting false alarms. This study presents an end-to-end explainable machine learning framework for credit card fraud detection, covering data preprocessing, class imbalance handling, model development, comparative evaluation, explainability, and deployment performance assessment. Logistic Regression, Random Forest, and XGBoost models were trained and evaluated using multiple complementary metrics, including precision, recall, F1-score, ROC-AUC, and confusion matrices. SMOTE was applied to address class imbalance in the training data. Among the evaluated models, Random Forest provided the most balanced precision-recall performance among the evaluated models. SHAP analysis was subsequently applied to the selected Random Forest model to provide both global and local explanations of model predictions and to identify the most influential features. The selected model was integrated into a Flask-based REST API and evaluated under repeated prediction requests. The load-testing experiment successfully processed 500 requests with no failures and achieved a throughput of 18.86 requests per second. The results demonstrate the feasibility of combining predictive performance, explainability, and deployment evaluation within a practical fraud detection framework.

Keywords: Credit Card Fraud Detection, Machine Learning, Random Forest, SMOTE, Explainable AI, SHAP, REST API, Deployment Performance Evaluation

1 Introduction

Credit card fraud has become an important challenge in modern financial systems as the increasing volume of digital transactions creates more opportunities for fraudulent activities. Fraudulent transactions can result in direct financial losses for customers and financial institutions while also affecting trust in digital payment systems. The rapid growth of electronic payment services has therefore increased the need for automated fraud detection techniques capable of identifying suspicious transactions efficiently and reliably. Machine learning (ML) has emerged as a promising approach because it can

learn patterns from historical transaction data and distinguish potentially fraudulent transactions from legitimate ones [1,2,6].

A major difficulty in credit card fraud detection is the highly imbalanced nature of transaction datasets. In typical fraud detection datasets, legitimate transactions greatly outnumber fraudulent transactions, causing conventional classification models to favor the majority class. Consequently, a model may achieve high overall accuracy while still failing to identify a significant proportion of fraudulent transactions. Previous research has therefore investigated several class imbalance handling techniques, including over-sampling, undersampling, and Synthetic Minority Oversampling Technique (SMOTE) [4,7,11].

In addition to class imbalance, selecting an appropriate machine learning model is important because different algorithms exhibit different capabilities when learning complex transaction patterns. Logistic Regression, Decision Tree, Random Forest, and XGBoost have all been investigated for credit card fraud detection, with ensemble-based approaches frequently demonstrating strong performance. Therefore, evaluating multiple models using metrics such as Precision, Recall, F1-score, and ROC-AUC is necessary to obtain a more reliable assessment of fraud detection performance [2–4,6].

Although machine learning models can provide effective fraud predictions, predictive performance alone does not explain why a particular transaction has been classified as fraudulent. This limitation is especially important in financial applications, where analysts and other stakeholders may need to understand the factors contributing to a model's decision. Explainable Artificial Intelligence (XAI) techniques can address this limitation by providing information about the contribution of individual features to model predictions. SHAP (SHapley Additive exPlanations) provides a systematic approach for interpreting machine learning predictions and can be used to obtain both global feature-importance information and explanations for individual predictions [12].

Another limitation in existing fraud detection research is that many studies concentrate primarily on the predictive performance of machine learning models while giving comparatively less attention to the practical deployment of those models. The transition from an experimental model to a usable fraud detection system requires additional components, including an application programming interface (API), persistent data storage, a user interface, containerized deployment, and performance evaluation under operational workloads. However, much of the existing literature continues to emphasize predictive performance, while practical system-level considerations such as explainability, API-based integration, deployment behavior, and resource utilization receive comparatively less attention within a single experimental framework [5,6,8,10].

To address these limitations, this research proposes an **end-to-end explainable machine learning framework for credit card fraud detection with deployment performance evaluation**. The framework combines comparative evaluation of Logistic Regression, Random Forest, and XGBoost with class imbalance handling,

comprehensive performance evaluation, and SHAP-based explainability. The selected machine learning model is subsequently integrated into a Flask-based RESTful API, while PostgreSQL is used for persistent storage and a Flutter Web application provides an interactive interface for transaction prediction, dashboard visualization, and prediction-history monitoring. Docker is used to containerize the backend infrastructure, providing a consistent and portable deployment environment.

The main contribution of this work is the integration of **machine learning evaluation, explainable artificial intelligence, application development, database management, containerized deployment, and production-oriented performance evaluation within a single fraud detection framework**. Rather than treating fraud detection solely as a classification problem, the proposed approach considers the complete lifecycle from transaction input and model prediction to data persistence, user interaction, explainability, and deployment performance. The framework therefore aims to bridge the gap between experimental machine learning research and practical fraud detection system development.

2 Related Work

Credit card fraud detection has been extensively studied using machine learning techniques because of the difficulty of identifying fraudulent transactions within highly imbalanced financial datasets. Existing research has investigated a variety of supervised learning algorithms, including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbors, Naive Bayes, and ensemble-based models. These approaches differ in their ability to capture complex transaction patterns and in their sensitivity to the severe imbalance between legitimate and fraudulent transactions [1–4,6,8,9].

Several studies have investigated ensemble learning techniques for improving fraud detection performance. Previous studies have compared Decision Tree, Random Forest, and XGBoost models for credit card fraud detection and reported strong performance from ensemble-based approaches [2–4,6,8]. Other studies have compared Logistic Regression, Decision Tree, XGBoost, and different sampling strategies, including random oversampling, random undersampling, and SMOTE [4,7,11]. Research comparing Logistic Regression, Support Vector Machine, Random Forest, and Artificial Neural Networks has also reported strong performance from Random Forest and neural-network-based approaches [3,6].

Class imbalance represents another major research challenge. Since fraudulent transactions constitute only a small proportion of the total transaction population, models trained directly on the original distribution may become biased toward legitimate transactions [4,6,7]. Consequently, researchers have investigated oversampling, undersampling, and hybrid approaches to improve minority-class detection. SMOTE is

particularly relevant because it generates synthetic minority-class examples rather than simply duplicating existing observations [7,11].

Despite these developments, much of the existing research primarily focuses on predictive performance. Comparatively fewer studies combine model evaluation with explainability and complete deployment architecture. In particular, the integration of SHAP-based explainability with REST APIs, persistent databases, containerized deployment, and production-oriented performance evaluation remains less explored [5,6,8,10].

The present research addresses this gap by combining comparative machine learning evaluation, class imbalance handling, SHAP-based explainability, and an end-to-end deployed architecture. The proposed framework integrates a machine learning prediction component with Flask-based REST services, PostgreSQL data persistence, Flutter Web interaction, and Docker-based deployment. In addition to evaluating classification performance, the study evaluates practical deployment characteristics such as API latency, throughput, CPU utilization, and memory consumption.

3 Proposed Methodology

The overall workflow of the proposed credit card fraud detection system is illustrated in Fig. 1. The framework integrates data preprocessing, exploratory analysis, class imbalance handling, machine learning model training and evaluation, model selection, SHAP-based explainability, and deployment components into an end-to-end fraud detection pipeline.

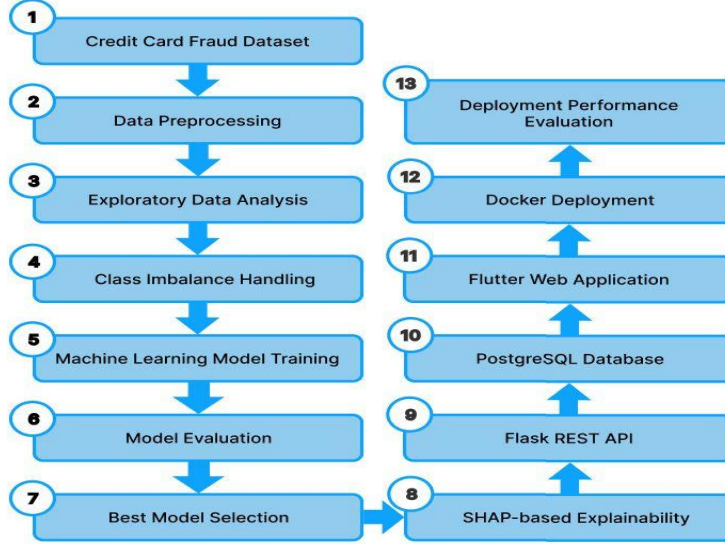


Fig. 1. Overall Framework of the Proposed Credit Card Fraud Detection System

3.1 Dataset and Preprocessing

The proposed framework begins with the preparation of transaction data for machine learning-based fraud detection. The data preprocessing stage is responsible for organizing the transaction records, identifying the target class, and preparing the input features for model training. Since the objective of the study is to distinguish fraudulent transactions from legitimate transactions, the transaction class is treated as the target variable, while the remaining relevant attributes are used as input features.

The preprocessing pipeline is designed to ensure that the data provided to the machine learning models is consistent and suitable for classification. Data preparation includes checking the structure of the dataset, handling the required transformations, and separating the available observations into the feature matrix and target vector. The resulting processed data is then used for model training and evaluation.

Particular attention is given to the distribution of the target classes because fraudulent transactions represent a minority class in credit card transaction data. This imbalance can significantly influence the learning process and may cause a model to favor legitimate transactions if the data is used without appropriate treatment.

3.2 Class Imbalance Handling

Class imbalance is a major challenge in credit card fraud detection because legitimate transactions substantially outnumber fraudulent transactions. A classifier trained directly on such data may achieve a high overall accuracy while performing poorly on the minority fraud class. Therefore, the proposed framework incorporates class imbalance handling as an important component of the model development process.

Different approaches to handling class imbalance can be considered, including oversampling, undersampling, and synthetic data generation techniques. In this study, SMOTE was applied to the training data to generate synthetic minority-class samples and reduce the effect of class imbalance during model training. The resampling procedure was applied only to the training portion of the dataset to avoid introducing synthetic observations into the evaluation data.

The objective of this stage is not simply to increase the number of minority-class observations, but to improve the ability of the trained models to identify fraudulent transactions while maintaining reliable performance on legitimate transactions.

3.3 Machine Learning Model Development

To investigate the effectiveness of different classification approaches, the proposed framework evaluates multiple supervised machine learning algorithms. Logistic Regression, Random Forest, and XGBoost are considered to provide a comparative assessment of linear, tree-based, ensemble, and gradient-boosting approaches.

Logistic Regression provides a baseline classification model and offers a relatively simple representation of the relationship between the input features and the probability of fraud. Random Forest combines multiple decision trees to improve predictive robustness, while XGBoost provides a gradient-boosting approach capable of modelling complex relationships within transaction data.

The models are trained using the prepared transaction data and evaluated using the same experimental framework. This enables a consistent comparison of their predictive capabilities and supports the selection of a suitable model for subsequent integration into the deployed fraud detection system.

3.4 Model Evaluation

Because credit card fraud detection involves a highly imbalanced classification problem, relying solely on accuracy does not provide a sufficiently informative assessment of model performance. The proposed framework therefore evaluates the candidate models using multiple classification metrics, including precision, recall, F1-score, and receiver operating characteristic area under the curve (ROC-AUC).

Precision measures the proportion of transactions predicted as fraudulent that are actually fraudulent, while recall measures the ability of the model to identify fraudulent transactions. The F1-score provides a balance between precision and recall. ROC-AUC is additionally used to assess the model's ability to distinguish between the two classes across different classification thresholds.

The comparative evaluation allows the selected model to be assessed not only in terms of overall predictive performance but also in terms of its effectiveness in identifying the minority fraudulent class.

3.5 Explainable AI Using SHAP

Predictive performance alone is insufficient for practical fraud detection because users may need to understand why a particular transaction has been classified as fraudulent. To address this requirement, the proposed framework incorporates Explainable Artificial Intelligence (XAI) using SHAP (SHapley Additive exPlanations).

SHAP provides a systematic approach for interpreting machine learning predictions by estimating the contribution of individual input features to a model's output. The framework uses SHAP to obtain feature-importance information and to provide explanations for individual transaction predictions.

The explanations provide an additional layer of transparency to the fraud detection process. Instead of presenting only a binary prediction, the system can associate a prediction with the features that contributed to the model's decision. This makes the resulting

fraud detection system more interpretable and potentially more useful for analysts who need to investigate suspicious transactions.

3.6 End-to-End System Architecture

The proposed framework extends the machine learning pipeline into an end-to-end deployable application. After model training and evaluation, the selected machine learning model is integrated into a Flask-based REST API that provides prediction services to the application layer. The API receives transaction information, processes the input using the trained model, and returns the corresponding fraud prediction and associated information.

PostgreSQL is used for persistent storage of application and prediction-related data. This enables transaction predictions and prediction history to be stored and retrieved for subsequent analysis. A Flutter Web application provides the user-facing interface through which users can interact with the fraud detection system, submit transactions for prediction, visualize relevant information, and monitor prediction history.

Docker is used to containerize the backend infrastructure and provide a consistent deployment environment. The resulting architecture therefore connects machine learning, explainability, REST-based services, persistent data storage, web interaction, and containerized deployment within a single fraud detection framework.

4 Experimental Setup

4.1 Experimental Environment

The experiments were conducted using a Python-based machine learning environment. The implementation used standard machine learning and data-processing libraries for data preparation, model training, evaluation, and explainability. The candidate classification models were trained and evaluated using the same experimental workflow to ensure a consistent comparison of their performance.

The developed system was additionally integrated with a Flask-based REST API, PostgreSQL database, and Flutter Web interface. This experimental setup enabled the evaluation of the machine learning component together with the application and data-management components of the proposed framework.

4.2 Model Training Configuration

The candidate models were trained using the preprocessed transaction dataset and the class-imbalance handling procedure described in Section 3. The evaluated models included Logistic Regression, Random Forest, and XGBoost. Each model was trained

using the same prepared data and evaluated using precision, recall, F1-score, and ROC-AUC.

After comparative evaluation, the best-performing model was selected for integration into the deployed fraud detection pipeline. SHAP-based explanations were subsequently incorporated to provide feature-level interpretation of the model predictions.

4.3 Deployment Environment

To evaluate the practical deployment of the proposed framework, the trained model was integrated into a Flask REST API. PostgreSQL was used for persistent storage, while the Flutter Web application provided the user interface for transaction prediction and prediction-history visualization. The backend infrastructure was containerized using Docker to provide a consistent and portable deployment environment.

In addition to predictive performance, the deployed system was evaluated using practical characteristics including API response latency, throughput, CPU utilization, and memory consumption. These measurements provide an assessment of the operational behavior of the framework beyond the classification metrics obtained during model evaluation.

5 Results and Performance Evaluation

The proposed fraud detection platform was evaluated from three complementary perspectives: **predictive performance, model interpretability, and deployment efficiency**. Logistic Regression, Random Forest, and XGBoost were compared using accuracy, precision, recall, F1-score, and ROC-AUC. The selected Random Forest model was subsequently analyzed using SHAP to interpret feature contributions and evaluated in its deployed environment using **API latency, load-testing throughput, and Docker resource utilization**.

5.1 Comparative Model Performance

Table 1 presents the performance of the three machine learning models evaluated for fraud detection.

Table 1. Comparative Performance of Machine Learning Models

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9746	0.0589	0.9184	0.1106	0.9712
Random Forest	0.9995	0.8602	0.8163	0.8377	0.9703
XGBoost	0.9969	0.3414	0.8673	0.4899	0.9767

The experimental evaluation demonstrates notable differences in the performance of the three machine learning models used for credit card fraud detection. Logistic Regression achieved an accuracy of 0.9746 and a high recall of 0.9184; however, its precision of 0.0589 and F1-score of 0.1106 indicate a large number of false-positive predictions. This suggests that although the model was effective at identifying fraudulent transactions, it generated a substantial number of incorrect fraud alerts.

Random Forest provided the most balanced precision-recall performance among the evaluated models, with an accuracy of 0.9995, precision of 0.8602, recall of 0.8163, and F1-score of 0.8377. The comparatively high precision indicates that the model was able to identify fraudulent transactions while substantially limiting false-positive predictions. These results make Random Forest the most suitable model among the evaluated classifiers for the deployed fraud detection platform.

XGBoost obtained the highest ROC-AUC value of 0.9767 and the highest recall of 0.8673. However, its precision of 0.3414 and F1-score of 0.4899 were considerably lower than those of Random Forest. Therefore, although XGBoost demonstrated strong discrimination and fraud detection capability, Random Forest provided a more balanced precision-recall performance for the intended application.

5.2 Model Explainability Analysis

The selected Random Forest model was further analyzed using SHAP (SHapley Additive exPlanations) to improve the interpretability of fraud detection decisions. The analysis was performed at both global and local levels to identify the transaction features that most strongly influenced model predictions and to examine feature contributions for an individual transaction.

The global SHAP analysis identified Feature 14, Feature 12, Feature 4, Feature 3, and Feature 10 as the most influential features in the model. Their relatively high mean absolute SHAP values indicate that these attributes had the greatest average impact on the model output across the evaluated transactions. The distribution of SHAP values also demonstrates that the influence of individual features varies across transactions, indicating that the Random Forest model combines multiple transaction characteristics when generating predictions.

Fig. 2 presents the global SHAP summary plot of the selected Random Forest model. The results show that Feature 14, Feature 12, Feature 4, Feature 3, and Feature 10 have the largest overall influence on the model output. The spread of SHAP values across the horizontal axis further indicates that the magnitude and direction of feature contributions vary among individual transactions.

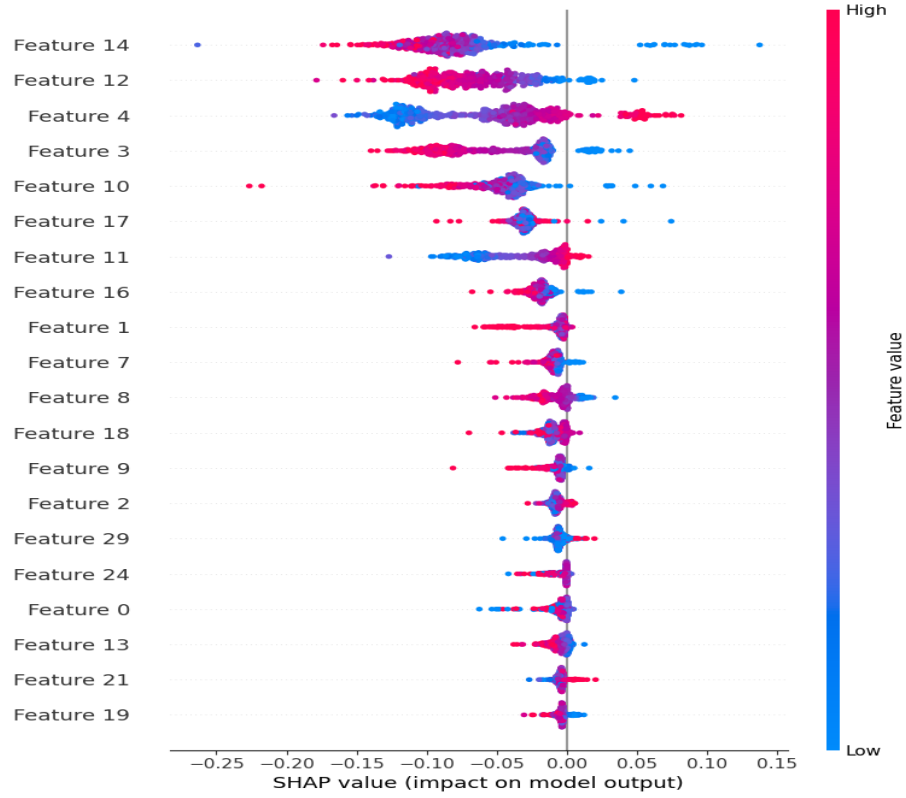


Fig. 2. SHAP Summary Plot of Global Feature Importance

In addition to global feature importance, a local SHAP analysis was performed to examine the contribution of individual features for a specific transaction. The local explanation provides a transaction-level decomposition of the model output, showing how individual feature contributions combine to produce the prediction relative to the expected model output.

Fig. 3 presents a representative local SHAP waterfall explanation for an individual transaction. The explanation shows the contributions of the leading features, including Feature 14, Feature 11, Feature 10, Feature 1, Feature 12, Feature 4, Feature 16, Feature 17, and Feature 3, while the remaining features are grouped as “21 other features.” The visualization demonstrates how the individual feature contributions collectively influence the model output for the selected transaction.

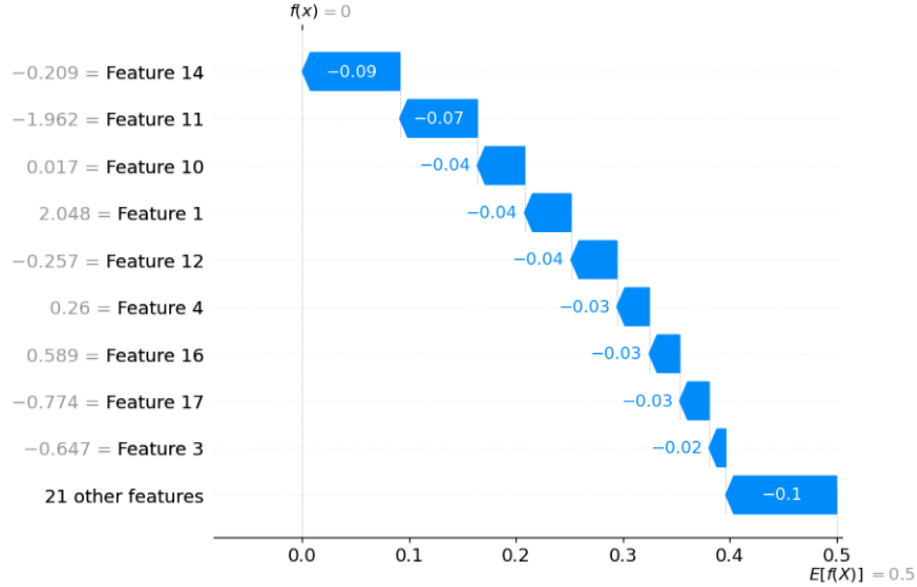


Fig. 3. Local SHAP Waterfall Explanation for a Representative Transaction

The global and local analyses therefore provide complementary forms of interpretability. The global analysis identifies the features that are influential across the evaluated transactions, whereas the local analysis illustrates how feature contributions combine for an individual prediction. Together, these analyses provide additional transparency into the decision-making behavior of the selected Random Forest model and can support analyst verification when investigating suspicious transactions.

5.3 Deployment Performance Evaluation

The deployed fraud detection service was evaluated in terms of **API response latency and Docker resource utilization** to assess the practical efficiency of the proposed system. The evaluation focused on repeated prediction requests to the Flask REST API and monitoring the computational resources consumed by the containerized backend during model inference.

The API latency experiment was performed by repeatedly sending representative transaction samples to the deployed prediction endpoint. The response time was measured from the transmission of each request until the corresponding prediction result was received. **The API latency distribution is shown in Fig. 4.** The latency distribution shows that most prediction requests were completed within approximately **45–60 ms**, with only a small number of higher-latency observations. These isolated values indicate occasional fluctuations while the overall inference service remained responsive and consistent.

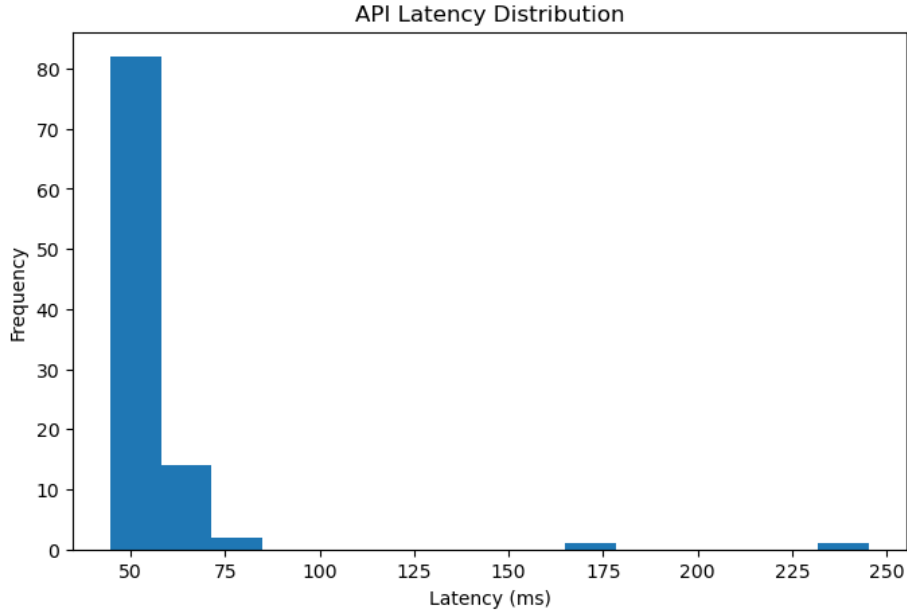


Fig. 4. Distribution of API Response Latency

To further evaluate the behavior of the deployed API under repeated requests, a load-testing experiment was conducted using 500 prediction requests. As shown in Table 2, all 500 requests were successfully processed with no failed requests. The experiment required 26.51 s in total and achieved a throughput of 18.86 requests per second. These results indicate that the deployed inference service was able to process the tested workload reliably while maintaining a consistent request-processing rate.

Table 2. Load Testing Results

Metric	Value
Total Requests	500
Successful Requests	500
Failed Requests	0
Execution Time (s)	26.51
Throughput (requests/s)	18.86

The resource utilization results further demonstrate the lightweight nature of the deployed inference service. As shown in Table 3, the observed average CPU utilization was **0.086978%**, while the peak CPU utilization reached **0.788200%**. The average and peak memory consumption were **330.87 MB** and **331.82 MB**, respectively. The relatively small difference between average and peak memory usage indicates stable memory consumption during repeated inference operations.

Table 3. Docker Resource Utilization Results

Metric	Measure Value
Average CPU Usage (%)	0.086978
Peak CPU Usage (%)	0.788200
Average Memory Usage (MB)	330.87
Peak Memory Usage (MB)	331.82

Overall, the deployment evaluation demonstrates that the proposed fraud detection service provides **low-latency prediction with stable resource consumption under the tested workload**. The combination of REST-based inference and Docker-based deployment therefore provides a practical foundation for integrating the trained Random Forest model into an operational fraud detection platform.

6 Discussion

The experimental results demonstrate that the proposed fraud detection platform provides a balanced combination of predictive performance, model interpretability, and deployment efficiency. The comparative evaluation showed that the Random Forest model achieved the most balanced performance among the evaluated classifiers, particularly in terms of precision, recall, and F1-score. Although XGBoost achieved a higher ROC-AUC and recall, its substantially lower precision and F1-score indicate that Random Forest provides a more suitable balance for the intended fraud detection application.

The results also highlight the importance of considering multiple evaluation metrics when assessing fraud detection models. The high accuracy values alone do not adequately describe model effectiveness in a highly imbalanced classification problem. Precision and recall provide additional insight into the trade-off between detecting fraudulent transactions and limiting false alerts. In this study, Logistic Regression achieved high recall but suffered from very low precision, whereas Random Forest provided substantially higher precision while maintaining competitive recall. This indicates that model selection for fraud detection should consider the operational consequences of false-positive and false-negative predictions rather than relying on accuracy alone.

The SHAP analysis further strengthens the practical value of the selected Random Forest model by providing insight into its decision-making process. The global analysis identified a relatively small group of influential features, including Feature 14, Feature 12, Feature 4, Feature 3, and Feature 10. The distribution of SHAP values also demonstrated that feature contributions vary across transactions, reflecting the non-linear relationships learned by the ensemble model. This interpretability is particularly valuable in financial applications where analysts may need to understand why a transaction was classified as potentially fraudulent.

From a deployment perspective, the API and Docker evaluations indicate that the proposed system can perform inference with low response latency while maintaining relatively stable resource consumption. Most API requests were completed within approximately 45–60 ms, while the Dockerized backend exhibited low CPU utilization and stable memory usage during the evaluation. These observations demonstrate that the proposed framework extends beyond offline machine learning experimentation by considering practical characteristics of a deployed prediction service.

Despite these results, several limitations should be acknowledged. The experimental evaluation was conducted using a publicly available credit card transaction dataset, and therefore the observed performance may not directly represent the characteristics of proprietary financial transaction streams. In addition, the deployment benchmark represents the tested workload and does not constitute a comprehensive assessment of large-scale production traffic, concurrent requests, or horizontal scalability. The feature names in the dataset are also anonymized, which limits the ability to associate the most influential SHAP features with directly interpretable financial attributes.

Overall, the findings indicate that effective fraud detection requires a combination of **predictive accuracy, balanced classification performance, interpretability, and deployment efficiency**. The proposed framework addresses these dimensions within a single end-to-end platform, providing a practical foundation for explainable and deployable credit card fraud detection.

7 Conclusion

This paper presented an end-to-end explainable credit card fraud detection platform that integrates machine learning, SHAP-based model interpretability, REST API deployment, Docker containerization, and a Flutter web application into a unified fraud detection framework. The proposed system was designed to address both predictive performance and practical deployment requirements for fraud detection applications.

A comparative evaluation of multiple machine learning algorithms demonstrated that the Random Forest classifier achieved the most balanced overall performance, providing high precision, strong F1-score, and competitive recall for fraud detection. Furthermore, SHAP explainability enabled transparent interpretation of model predictions by identifying the transaction features that contributed most significantly to fraud classification. This improves analyst confidence and supports more interpretable AI-assisted decision making.

The deployment experiments further showed that the proposed platform delivers low API response latency together with stable CPU and memory utilization, indicating that accurate fraud prediction can be achieved without excessive computational overhead. These results demonstrate the practical feasibility of combining predictive analytics with lightweight deployment architecture for low-latency fraud detection services.

Future work will focus on evaluating the framework using larger real-world financial transaction datasets, investigating advanced ensemble and deep learning architectures, and extending the deployment infrastructure to support higher levels of scalability, streaming transaction processing, and continuous model updating for production financial systems.

Acknowledgments. The author would like to sincerely thank **Saloni Thakur** for her assistance in designing and preparing the diagrams and figures presented in this research work.

Disclosure of Interests. The author declares that there are no competing interests relevant to the content of this article.

References

1. V. N. Dornadula and S. Geetha: Credit card fraud detection using machine learning algorithms. *Procedia Computer Science* **165**, 631–641 (2019).
2. J. K. Afriyie *et al.*: A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions. *Decision Analytics Journal* **6**, 100163 (2023).
3. Q. S. Mirhashemi, N. Nasiri, and M. R. Keyvanpour: Evaluation of supervised machine learning algorithms for credit card fraud detection: A comparison. In: *2023 9th International Conference on Web Research (ICWR)*, pp. 247–252. IEEE (2023).
4. M. G. Wijaya *et al.*: Comparative analysis of machine learning algorithms and data balancing techniques for credit card fraud detection. *Procedia Computer Science* **245**, 677–688 (2024).
5. T. Albalawi and S. Dardouri: Enhancing credit card fraud detection using traditional and deep learning models with class imbalance mitigation. *Frontiers in Artificial Intelligence* **8**, 1643292 (2025).
6. F. K. Alarfaj *et al.*: Credit card fraud detection using state-of-the-art machine learning and deep learning algorithms. *IEEE Access* **10**, 39700–39715 (2022).
7. E. Ileberi, Y. Sun, and Z. Wang: Performance evaluation of machine learning methods for credit card fraud detection using SMOTE and AdaBoost. *IEEE Access* **9**, 165286–165294 (2021).
8. A. R. Khalid *et al.*: Enhancing credit card fraud detection: An ensemble machine learning approach. *Big Data and Cognitive Computing* **8**(1), 6 (2024).
9. S. Mittal and S. Tyagi: Performance evaluation of machine learning algorithms for credit card fraud detection. In: *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, pp. 320–324. IEEE (2019).
10. F. Carcillo *et al.*: Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences* **557**, 317–331 (2021).
11. N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer: SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* **16**, 321–357 (2002).
12. S. M. Lundberg and S.-I. Lee: A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems* **30**, pp. 4765–4774 (2017).